

Web Fundamentals

How the internet actually works — the journey from typing a URL to seeing pixels on your screen.

WHAT WE COVER

The Internet

Clients & Servers

DNS

TCP/IP

HTTP

Before we write a single line of HTML, it pays to know what happens behind the scenes when you open a website. This handout builds the mental model of the web that the rest of the course stands on — the machines involved, how they find each other, and the rules they use to talk.

Web Fundamentals

Every website you have ever opened involved a short, invisible conversation between computers. Understanding that conversation once makes everything in front-end development easier to reason about later.

ON THIS HANDOUT

1. The internet vs. the web
2. Clients, servers & the nodes in between
3. Addresses: IP & ports
4. DNS — the internet's phonebook
5. TCP/IP — moving data reliably
6. HTTP — the language of the web
7. Putting it together: what happens when you press Enter
8. Key terms recap

01 The internet vs. the web

People use these words as if they mean the same thing. They don't.

The **Internet** is the global *network of networks* — the physical plumbing of cables, fibre, Wi-Fi, and cell towers that connects billions of devices worldwide. The **World Wide Web** is just *one service* that runs on top of that plumbing: the system of websites you visit with a browser.

Email, video calls, and online games are other services that use the same internet but are not “the web.”

ANALOGY

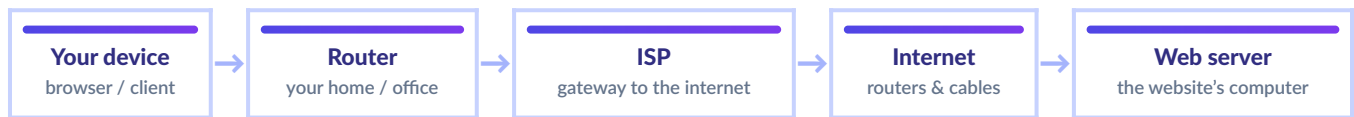
Think of the **internet** as a country's road and postal network, and the **web** as one kind of delivery van that drives on those roads. The roads carry many kinds of traffic; the web is only one of them.

02 Clients, servers & the nodes in between

The web runs on a simple pattern called the **client-server model**.

- A **client** asks for things. Your browser (Chrome, Firefox, Safari) is a client — it *requests* web pages on your behalf.
- A **server** answers. It is an always-on computer that stores a website and *serves* it to anyone who asks.

A **node** is any device connected to the network. Between your client and the server sits a small chain of nodes, each with one job:



A request hops through several nodes on its way to the server – and the reply travels back the same way.

Node	What it does
Client	The device and browser that make the request (your laptop or phone).
Router	Directs traffic between your local network and the wider internet.
Modem	Converts your network's signal to and from the format your ISP's line uses.
ISP	Internet Service Provider – the company that connects you to the rest of the internet.
Server	The always-on computer that stores the website and responds to requests.

NOTE

The word **server** can mean two things: the physical machine, or the software running on it that answers requests. Usually the context makes it clear.

03 Addresses: IP & ports

For one node to send data to another, it needs the other's address. On the internet that address is an **IP address** – a unique number identifying a device on the network.

- **IPv4** – the older format, four numbers 0–255, e.g. `142.250.190.78`. There are about 4.3 billion of these... which the world has run out of.
- **IPv6** – the newer, much larger format, e.g. `2607:f8b0:4005:80a::200e`, giving a practically unlimited supply.

Your devices at home share one **public IP** (given by your ISP) while each has its own **private IP** inside your network. Your router juggles the two so replies find the right device.

A single machine can run many services at once, so it also uses **ports** – numbered doors on that machine. The web uses two by default:

Port	Used for
80	HTTP – unencrypted web traffic
443	HTTPS – encrypted web traffic (the padlock)
53	DNS – name lookups

ANALOGY

An **IP address** is like a building's street address; a **port** is the specific office inside that building you want to reach.

04 DNS — the internet's phonebook

Humans remember names like `google.com`; machines route data using IP addresses. The **Domain Name System (DNS)** is the service that translates one into the other.

When you enter an address, a **DNS resolver** works through a hierarchy of servers until it finds the answer:

- 1 **Your browser & OS check their cache.** If they looked this name up recently, the IP is already known and the search stops here.
- 2 **The recursive resolver is asked.** Usually run by your ISP (or a public one like `8.8.8.8`). It does the legwork for you.
- 3 **A root server** points the resolver toward the servers responsible for the ending, e.g. `.com`.
- 4 **A TLD server** (for `.com`) points to the *authoritative* name server for `google.com`.
- 5 **The authoritative server** returns the actual IP address.
- 6 **The resolver caches the answer** and hands it back to your browser, which can now connect.

TIP

Each answer comes with a **TTL** (time-to-live) that says how long it may be cached. That is why visiting a site the second time feels faster — the lookup was skipped.

05 TCP/IP — moving data reliably

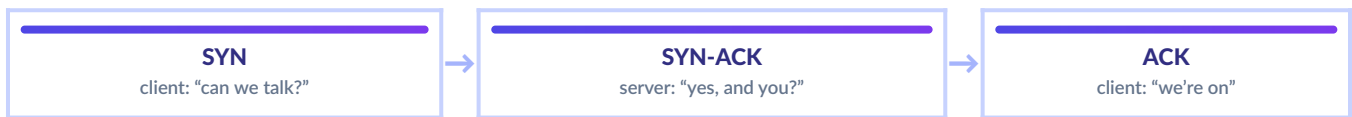
Once your browser knows the server's IP, it still needs a dependable way to send and receive data across an unreliable, crowded network. That job belongs to the **TCP/IP** family of protocols.

Data is never sent as one big lump. It is broken into small **packets** that travel independently and are reassembled at the other end.

The two workhorses

- **IP (Internet Protocol)** handles *addressing and routing* — getting each packet from one node to the next, hop by hop, toward its destination. It is “best effort”: fast, but it does not guarantee delivery or order.
- **TCP (Transmission Control Protocol)** adds *reliability* on top of IP. It numbers packets, confirms they arrived, re-sends anything lost, and puts them back in the right order.

Before sending data, TCP opens a connection with a quick **three-way handshake**:



The TCP three-way handshake establishes a reliable connection before any data flows.

Its faster cousin, **UDP**, skips the handshake and the guarantees. That trade-off suits jobs where speed beats perfection — live video, calls, and games.

The layers, simplified

Networking is organised into layers, each building on the one below. You rarely touch the lower ones, but it helps to know they exist:

Layer	Job	Examples
Application	The service you actually use	HTTP, DNS
Transport	Reliable (or fast) delivery between programs	TCP, UDP
Internet	Addressing & routing across networks	IP
Link	The physical hop to the next device	Wi-Fi, Ethernet

ANALOGY

Imagine mailing a long book by posting one page per envelope. IP is the postal system carrying each envelope; TCP is the friend at the other end who checks every page arrived and staples them back in order, asking you to re-send any that got lost.

06 HTTP — the language of the web

With a reliable connection open, the client and server finally talk using **HTTP (HyperText Transfer Protocol)** — the set of rules for the web. It follows a strict **request** → **response** pattern: the client asks for one thing, the server sends one reply.

A request

Every request states a **method** (what you want to do), a **path** (what you want), and some **headers** (extra details):

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Chrome/120
Accept: text/html
```

HTTP

A response

The reply starts with a **status code**, adds its own headers, then the **body** — here, the HTML of the page:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1256
```

HTTP

```
<!DOCTYPE html>
<html> ... </html>
```

Common methods

Method	Meaning
GET	Read / fetch a resource (the default when you visit a page)
POST	Send new data to the server (submit a form, sign up)
PUT	Replace an existing resource
PATCH	Update part of a resource
DELETE	Remove a resource

Status codes

The first digit tells you the category at a glance:

Range	Meaning	You'll see
2xx	Success	200 OK
3xx	Redirect — look elsewhere	301 Moved
4xx	You made a mistake	404 Not Found
5xx	The server hit an error	500 Server Error

WATCH OUT

HTTP is **stateless** — each request stands alone and the server remembers nothing between them. Sites use cookies and sessions (a later topic) to recognise you across requests.

Finally, **HTTPS** is just HTTP wrapped in **TLS** encryption. It scrambles the traffic so nobody in between can read or tamper with it — that is what the padlock in the address bar means. Today it is the norm for essentially every site.

07 Putting it together: what happens when you press Enter

Here is the whole journey, start to finish, when you type an address and hit `Enter`:

- DNS lookup.** The browser turns the domain name into an IP address (Section 4).

- 2 **TCP connection.** It opens a reliable connection to that IP with the three-way handshake (Section 5).
- 3 **TLS handshake.** For an `https://` site, both sides agree on encryption keys so the traffic is private.
- 4 **HTTP request.** The browser sends a `GET` request for the page (Section 6).
- 5 **Server responds.** It processes the request and returns an HTTP response containing the HTML.
- 6 **The browser renders.** It reads the HTML and, as it goes, makes more requests for the CSS, JavaScript, images, and fonts the page needs.
- 7 **The page appears** — assembled from all those pieces, ready for you to use.

KEY IDEA

That last step — the browser reading HTML and pulling in CSS and images — is exactly where the rest of this course begins. Next up: writing the HTML and CSS the server sends back.

08 Key terms recap

Term	In one line
Internet	The global network of networks — the physical plumbing.
Web	One service on the internet: websites delivered over HTTP.
Client / Server	The asker (your browser) and the answerer (the site's computer).
Node	Any device connected to the network.
IP address	A unique number identifying a device on the network.
Port	A numbered door on a machine for a specific service.
DNS	Translates human names into IP addresses.
Packet	A small chunk of a larger message, sent independently.
TCP / IP	Reliable delivery (TCP) built on addressing & routing (IP).
HTTP / HTTPS	The request-response rules of the web; HTTPS adds encryption.

Up next — Day 1: HTML & CSS Fundamentals

Now that you know what the browser receives, you'll learn to write it: the HTML that structures a page and the CSS that styles it. Keep the two cheat sheets from Day 1 next to you as you practise.